中大愛心綠色腳踏車 NCU Green Love Bikes

自建腳踏車損耗模型 × Claude AI 混合檢測

從收集照片、標註、訓練，到與 Claude 合併判斷的完整教學

你將學會

- 定義腳踏車損耗等級並收集訓練照片
- 用 Claude 自動預標註，再人工覆核
- 用 Teachable Machine 訓練自己的影像模型
- 把模型放進網站，與 Claude 結果合併判斷
- 評估準確度、調整門檻，並持續改進模型

版本 1.0　2026 年 11 月　|　學生專案教學文件

目錄

第 1 章 整體架構 2
第 2 章 事前準備 4
第 3 章 定義損耗等級 5
第 4 章 收集訓練照片 6
第 5 章 用 Claude 協助標註 7
第 6 章 訓練自己的模型 9
第 7 章 把模型放進網站 11
第 8 章 與 Claude 合併判斷 14
第 9 章 測試與調整 17
第 10 章 常見問題與上線檢查 19

閱讀方式

第 1–3 章是觀念與準備，第 4–6 章是訓練自己的模型，第 7–8 章是放進網站並與 Claude 合併，第 9–10 章是測試、調整與疑難排解。第一次做建議依序閱讀；每章開頭都會說明這一章要完成什麼。

第 1 章 整體架構

先搞懂兩個模型各自擅長什麼，以及它們怎麼合作。

學生申請回收廢棄腳踏車時，要上傳 3 張照片（正面、側面、損壞處特寫）。我們要讓系統自動判斷這台車的**損耗率**，以及是否達到**合格門檻**。為了兼顧準確度、成本與可解釋性，我們同時使用兩個模型：

	① 自建模型（Teachable Machine）	② Claude 視覺 AI
怎麼來的	用我們自己拍、自己標註的照片訓練	Anthropic 提供的大型模型，不需訓練
在哪裡執行	學生的瀏覽器裡	Cloudflare Worker（後端）呼叫 Claude API
費用	免費	每次檢測約 US$0.03–0.06
速度	不到 1 秒	約 10–30 秒
擅長	我們校園常見車款的整體損耗等級	逐部位分析、寫出原因、辨識沒看過的狀況（電動車、非單車、照片模糊）
弱點	沒學過的狀況會亂猜；不會說明原因	要花錢、需要網路；偶爾判斷不穩定

兩者互補：自建模型提供快速、免費、針對我們資料的判斷；Claude 提供詳細說明與「看過大世面」的常識。兩者意見一致時自動採用，不一致時交給技師，這樣可以大幅減少誤判。

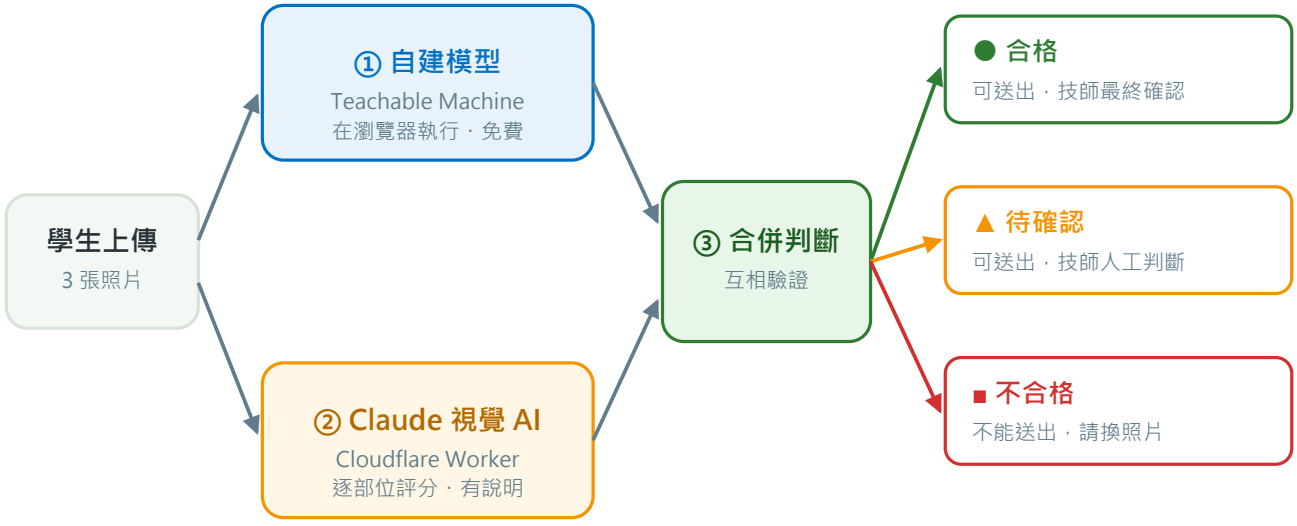


圖 1-1　混合檢測架構：兩個模型各自判斷，再依規則合併

合併判斷的結果分三種

結果	意思	網站行為
● 合格	兩個模型都認為車況達到門檻	可以送出申請，由技師做最終確認
▲ 待確認	兩個模型意見不同，或其中一個無法判斷	可以送出申請，由技師人工判斷
■ 不合格	兩個模型都認為不合格，或 Claude 發現車架斷裂、電動車等硬性問題	不能送出，請學生換照片或換一台車

資料飛輪：讓模型越用越準

自建模型的準確度取決於訓練照片。網站上線後，每一筆申請都會經過技師最終審核，技師的判斷就是最可靠的「正確答案」。定期把這些照片加回訓練資料重新訓練，模型就會越來越懂我們校園的腳踏車。

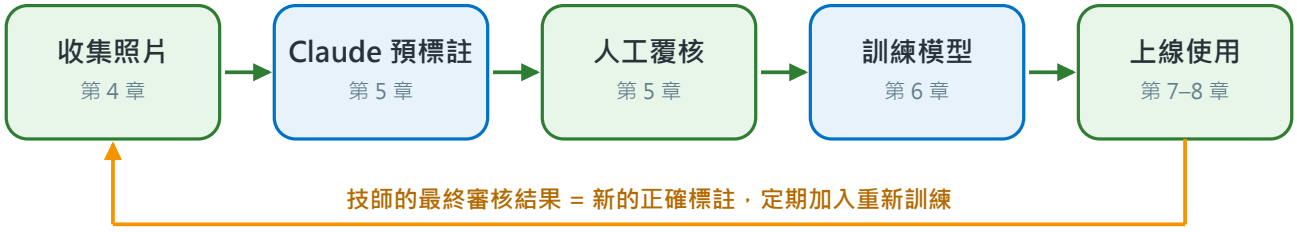


圖 1-2　資料飛輪：系統用得越久，自建模型越準

第 2 章 事前準備

準備帳號、軟體與資料夾。這一章大約需要 30 分鐘。

項目	用途	取得方式
Google 帳號	使用 Teachable Machine 訓練模型（可選擇存到 Google 雲端硬碟）	任何 Google 帳號皆可
Python 3.10 以上	執行 Claude 預標註程式	python.org 下載；安裝時勾選 Add Python to PATH
Node.js（LTS 版）	部署 Claude 後端（Cloudflare Worker）	nodejs.org 下載
Cloudflare 帳號	免費架設 Claude 後端	dash.cloudflare.com 註冊
Claude API Key	呼叫 Claude；記得設定每月花費上限	platform.claude.com → API Keys → Somewhere else
本專案程式碼	網站、後端、標註程式	git clone 專案到 C:\Projects\final-project

專案資料夾結構

完成本教學後，專案會長這樣（★ 是本教學會新增或修改的部分）：

```
final-project/
├── website/                  網站
│   ├── Eng.html
│   ├── script.js             已內建自建模型與合併判斷
│   └── model/                ★ 放 Teachable Machine 匯出的模型檔
│       ├── model.json
│       ├── metadata.json
│       └── weights.bin
├── ai-inspector/             Claude 後端（Cloudflare Worker，已完成）
└── ml-training/              ★ 訓練資料與工具
    ├── label_with_claude.py  Claude 預標註程式（專案已附）
    ├── raw/                  還沒標註的照片
    ├── labeled/A ~ X/        標註後的照片（依等級分資料夾）
    ├── test/A ~ X/           保留的測試照片（不拿來訓練）
    └── labels.csv            標註紀錄
```

API Key 是密碼，絕對不要外流

不要把 API Key 寫進程式碼、上傳到 GitHub 或貼到聊天室。只用環境變數（第 5 章）或 wrangler secret（部署後端）設定。萬一外流，立刻到 Console 刪除並重新建立。

第 3 章 定義損耗等級

訓練前最重要的一步：大家對「什麼算嚴重」要有一致的標準。

模型只能學會我們教它的東西。如果標註的人標準不一，模型也會學得亂七八糟。所以我們先定義 5 個類別，並讓 Claude 後端、自建模型、技師都使用同一套標準。

類別	名稱	代表分數	判斷依據	回收結果
A	良好	95	接近全新，只有輕微使用痕跡	合格
B	輕微磨損	80	一般刮傷、輕微表面鏽、輪胎或座墊小磨損	合格
C	中度磨損	60	明顯老舊，至少一個零件需更換（輪胎磨平、鏈條生鏽、煞車線鬆脫、座墊破損）	看門檻（預設 70，不合格）
D	嚴重損壞	30	車架斷裂/彎曲、輪框嚴重變形、大面積鏽蝕、缺少主要零件	不合格
X	無法使用	—	照片中沒有腳踏車、電動車、兒童車、照片太模糊或太暗	交給 Claude/技師判斷

為什麼要有 X 類別？

模型只會從「學過的類別」裡選一個答案。如果沒有 X，學生上傳一張貓的照片，模型也會硬說它是 A 或 B。加入 X 類別，模型就能說出「這張我判斷不了」。

「代表分數」怎麼用？

自建模型輸出的是每個類別的機率，例如 A 10%、B 60%、C 25%、D 5%。我們用機率乘上代表分數再加總，就得到 0–100 的**車況分數**：0.10×95 + 0.60×80 + 0.25×60 + 0.05×30 = 74 分，**損耗率 = 100 - 74 = 26%**。這和 Claude 後端的計分方式一致，兩邊分數可以直接比較。

> **先跟技師開一次會**
>
> 拿 20 張真實照片，請技師和組員各自分類，再比較答案。意見不同的照片拿出來討論，把結論補進上表的「判斷依據」。這一步做好，後面的標註會快很多，模型也會準很多。

第 4 章 收集訓練照片

照片的品質與多樣性，決定了模型的上限。

項目	最少	建議	說明
每個類別的照片數	50 張	150 張以上	越多越好；各類別數量盡量接近，不要 A 有 300 張、D 只有 20 張
不同的車	每類 15 台	每類 40 台以上	同一台車拍很多張，模型會記住「那台車」而不是「損耗特徵」
測試照片	每類 10 張	每類約 20%	放在 test/，絕對不拿去訓練，用來檢驗模型

拍攝規範

- **和學生上傳的照片一樣**：用手機拍，包含正面、側面、損壞處特寫三種角度。模型在什麼樣的照片上訓練，就只會判斷那樣的照片。
- **背景與光線要多樣**：白天、陰天、室內、車棚、草地都要有。如果 D 類全在修理站拍、A 類全在停車場拍，模型會學成「看背景」。
- **車子佔畫面大部分**：不要拍太遠；損壞處特寫要清楚對焦。
- **每張照片只有一台主要的車**：一排停在一起的車很難判斷是哪一台。
- **X 類別要刻意收集**：模糊照、太暗的照片、電動車、兒童車、只有地面或牆壁的照片，各拍一些。

照片來源

- 校園內的廢棄車（拍攝前先和總務處、修理站確認，不要移動或破壞車輛）
- 修理站正在維修的車，以及同學、教職員自願提供的車
- 網站上線後，學生上傳的申請照片（經技師審核後，就是最好的訓練資料）

> **隱私與版權**
>
> 避免拍到路人臉孔、車牌或個人物品；拍到的話請裁切掉。不要從網路上隨意下載別人的照片當訓練資料，可能有版權問題。

檔案整理

把所有還沒分類的照片放進 **ml-training/raw/**。檔名不用改，但不要有重複檔名。建議每次拍完就整理，並在檔名或筆記中記錄是哪一台車，方便之後分出測試照片。

第 5 章 用 Claude 協助標註

讓 Claude 先分類，人只要檢查和修正，速度快好幾倍。

手動把幾百張照片一張張分類很花時間。我們讓 Claude 先依第 3 章的標準給每張照片一個等級，再由人工覆核。**Claude 的標註只是草稿**，最後一定要人看過，因為模型會完全照著標註學習。

步驟 1：安裝套件

PowerShell

```
cd C:\Projects\final-project\ml-training
pip install anthropic pillow
```

步驟 2：設定 API Key（只在這個視窗有效，關掉就消失，比較安全）

PowerShell

```
$env:ANTHROPIC_API_KEY = "貼上你的 API Key"
```

步驟 3：執行預標註

PowerShell

```
python label_with_claude.py
```

程式會讀取 raw/ 的每張照片，縮小後送給 Claude，結果寫到 **labeled/A ~ X/** 資料夾與 **labels.csv**。中斷後重新執行會自動略過已完成的照片。執行畫面類似：

執行結果範例

```
[1/240] IMG_0012.jpg → B  輪胎略有磨損，車架完整無變形
[2/240] IMG_0013.jpg → D  前輪輪框明顯彎曲變形
[3/240] IMG_0014.jpg → X  照片過暗，無法判斷車況
...
本次標註數量：  A: 31  B: 88  C: 72  D: 35  X: 14
```

程式的核心部分

完整程式在 **ml-training/label_with_claude.py**。核心是請 Claude 依固定格式（JSON Schema）回答，確保每次都只會得到 A–X 其中一個等級：

label_with_claude.py（節錄）

```
response = client.beta.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    betas=["server-side-fallback-2026-07-01"],   # Claude 拒答時自動改用備援模型
    fallbacks="default",
    system=SYSTEM_PROMPT,                          # 第 3 章的等級定義
    output_config={
        "effort": "medium",
        "format": {"type": "json_schema", "schema": SCHEMA},  # 只能回答 A/B/C/D/X
    },
    messages=[{"role": "user", "content": [
        {"type": "image", "source": {"type": "base64",
                                     "media_type": "image/jpeg", "data": photo_b64}},
        {"type": "text", "text": "Grade this photo."},
    ]}],
)
```

步驟 4：人工覆核（最重要！）

1. 打開 labeled/ 的每個資料夾，用「大圖示」檢視，快速掃過每張照片。
2. 標錯的照片直接拖到正確的資料夾，並把 labels.csv 該列的 reviewed 改成 yes。
3. 拿不準的照片先放到 labeled/unsure/，開會時和技師討論，或乾脆不用。
4. 記錄 Claude 和人工判斷不同的比例。如果超過 30%，代表第 3 章的標準需要寫得更清楚。

步驟 5：分出測試照片

從每個類別挑出約 20% 的照片，移到 **test/** 對應的資料夾。**同一台車的所有照片要放在同一邊**（全部在 labeled 或全部在 test），否則測試成績會虛高。

> **費用估算**
>
> 每張照片約 US$0.01–0.02，500 張約 US$5–10。實際費用以 Claude Console 的用量頁為準。建議先放 20 張試跑，確認結果合理再跑全部。

第 6 章 訓練自己的模型

用 Google Teachable Machine，不用寫程式就能訓練影像分類模型。

Teachable Machine 使用「遷移學習」：它已經有一個看過上百萬張圖片的基礎模型（MobileNet），我們只要教它最後一步「怎麼分辨我們的 5 個類別」。所以只需要幾百張照片、幾分鐘就能訓練完成。

步驟 1：建立專案

1. 用瀏覽器開啟 teachablemachine.withgoogle.com，按 **Get Started**。
2. 選 **Image Project** → **Standard image model**。

步驟 2：建立 5 個類別並上傳照片

1. 把預設的 Class 1、Class 2 改名為 **A**、**B**，再按 **Add a class** 新增 **C**、**D**、**X**。
2. 每個類別按 **Upload** → **Choose images from your files**，選取 labeled/ 對應資料夾裡的全部照片。
3. **不要**上傳 test/ 的照片。

> **類別名稱一定要用英文 A、B、C、D、X**
> 網站程式會用這幾個名稱計算分數，名稱不同就會算錯。中文名稱在網站上另外顯示即可。

步驟 3：訓練

按 **Advanced** 展開進階設定，第一次建議使用預設值：

設定	建議值	說明
Epochs	50	整份資料要看幾遍。準確度還在上升就增加到 80–100；測試很差但訓練很好代表看太多遍（過度擬合），就減少
Batch Size	16	每次一起學幾張，維持預設即可
Learning Rate	0.001	學習速度，維持預設即可

按 **Train Model**，訓練時不要切換分頁（瀏覽器會暫停訓練）。完成後，右側的 **Preview** 可以切換成 **File**，上傳 test/ 裡的照片試試看。

步驟 4：查看準確度

在 Advanced 裡按 **Under the hood**，可以看到：**Accuracy per class**（每個類別的準確率）與 **Confusion Matrix**（混淆矩陣，看哪兩個類別最常搞混）。第 9 章會教你怎麼解讀。

步驟 5：匯出模型

1. 按右上角 **Export Model** → **Tensorflow.js** 分頁 → 選 **Download** → 按 **Download my model**。
2. 解壓縮下載的 zip，會得到 **model.json**、**metadata.json**、**weights.bin** 三個檔案。
3. 在 website/ 裡建立 **model** 資料夾，把三個檔案放進去。
4. 回到 Teachable Machine 按左上角選單 → **Save project to Drive**，下次可以接著加照片重新訓練。

> **準確度不夠時優先做的事**
>
> 1. 補照片給準確率最低的類別 2. 檢查那個類別有沒有標錯的照片 3. 讓背景與光線更多樣。調整 Epochs 等參數的效果，通常遠不如改善資料。

第 7 章 把模型放進網站

網站已經內建自建模型的程式，你只要放入模型檔。

程式已經寫好了

網站的 website/script.js 已經內建本章與第 8 章的程式。只要把 Teachable Machine 匯出的三個檔案放進 website/model/，網站就會自動開始使用自建模型；資料夾是空的時候，網站會自動略過自建模型，只使用 Claude。本章後半段說明程式怎麼運作，方便你調整設定或寫專題報告。

步驟 1：放入模型檔

1. 解壓縮第 6 章從 Teachable Machine 下載的 zip。
2. 把 **model.json**、**metadata.json**、**weights.bin** 三個檔案複製到 **website/model/**（資料夾不存在就自己建立）。
3. 檔名不要改。把網站上傳到正式網址時，model 資料夾也要一起上傳。

```
website/
├── Eng.html
├── script.js
└── model/
    ├── model.json
    ├── metadata.json
    └── weights.bin
```

步驟 2：檢查設定

所有設定都在 **website/script.js** 最上方：

website/script.js — 設定

```
// 自建模型（Teachable Machine 匯出的 model.json / metadata.json / weights.bin 放在這個資料夾）
// 資料夾裡沒有模型時會自動略過，只使用 Claude
const LOCAL_MODEL_URL = "model/";
const TFJS_URL = "https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@1.3.1/dist/tf.min.js";
const TM_IMAGE_URL = "https://cdn.jsdelivr.net/npm/@teachablemachine/image@0.8.5/dist/teachablemachine-image.
    min.js";
const GRADES = ["A", "B", "C", "D", "X"];              // 模型的類別名稱必須是這 5 個
const GRADE_SCORE = { A: 95, B: 80, C: 60, D: 30 };    // 各等級的代表分數
const PASS_SCORE = 70;                                  // 必須和 ai-inspector/wrangler.toml 的 PASS_SCORE
    一致
const LOCAL_MIN_CONFIDENCE = 0.6;                       // 自建模型信心低於這個值就不採用
const CLAUDE_WEIGHT = 0.6;                              // 最終分數 = Claude × 60% + 自建模型 × 40%
```

設定	意思	什麼時候要改
LOCAL_MODEL_URL	模型檔所在的資料夾	模型放在其他位置時；設成空字串 "" 可以暫時停用自建模型
GRADE_SCORE	各等級的代表分數（第 3 章）	和技師討論後覺得某個等級分數不合理時
PASS_SCORE	合格門檻	要更嚴格或更寬鬆時；必須和 ai-inspector/wrangler.toml 的 PASS_SCORE 一起改
LOCAL_MIN_CONFIDENCE	自建模型的最低信心	模型很準時可以降低，讓它更常參與判斷
CLAUDE_WEIGHT	最終分數中 Claude 的比重	自建模型測試準確率超過 85% 後，可以改成 0.5

步驟 3：測試

1. 在 website 資料夾執行 **python -m http.server 5500**，用瀏覽器開啟 http://localhost:5500（不能直接雙擊 HTML）。
2. 用學生帳號登入，到「會員專區」→「前往填寫回收申請單」，上傳 3 張照片並送出。
3. 結果視窗出現「自建模型」區塊，顯示 A–X 各等級的機率，就代表成功。
4. 沒有出現時，按 F12 打開主控台查看錯誤訊息（第 10 章有常見錯誤的解法）。

程式怎麼運作

第一次需要判斷時，網站才會載入 TensorFlow.js 與 Teachable Machine 函式庫（約 1MB），所以不會拖慢其他頁面。接著模型分別判斷 3 張照片，把機率平均後換算成車況分數：

website/script.js — runLocalModel()

```
// 回傳 null 代表沒有自建模型
async function runLocalModel(photos) {
    const model = await loadLocalModel();
    if (!model) return null;
    const perPhoto = [];
    for (const p of photos) {
        const preds = await model.predict(await dataUrlToImage(p)); // [{ className, probability }, ...]
        perPhoto.push(Object.fromEntries(preds.map(x => [x.className, x.probability])));
    }
    // 3 張照片的機率取平均
    const probs = {};
    GRADES.forEach(g => { probs[g] = perPhoto.reduce((s, p) => s + (p[g] || 0), 0) / perPhoto.length; });
    const topClass = GRADES.reduce((a, b) => (probs[a] >= probs[b] ? a : b));
    // 只用 A~D 計算分數（扣掉 X 的機率後重新分配）
    const bikeProb = 1 - probs.X;
    const score = bikeProb > 0.001
```

```
        ? Math.round(["A", "B", "C", "D"].reduce((s, g) => s + probs[g] * GRADE_SCORE[g], 0) / bikeProb)
        : 0;
    return { probs, topClass, confidence: probs[topClass], notBike: topClass === "X", score, wearRate: 100 -
        score };
}
```

為什麼要 3 張照片取平均？

單一張照片可能剛好拍不到損壞處。3 個角度的機率平均後，結果比較穩定；例如正面看起來是 B，但特寫照拍到斷裂（D 機率很高），平均後分數就會被拉低。

第 8 章 與 Claude 合併判斷

兩個模型同時執行，依規則合併成最終結果。

合併規則

規則由上往下檢查，符合第一條就停止。設計原則是：**Claude 對「硬性問題」有否決權**（因為自建模型沒學過電動車、車架斷裂這類少見狀況），**其他情況兩者都同意才自動決定**。

順序	情況	結果
1	Claude 發現硬性問題：非單車、電動車、兒童車、車架斷裂、輪框嚴重變形	■ 不合格
2	自建模型認為是 X（無法使用），且信心 ≥ 70%	▲ 待確認
3	Claude 自己判定「待確認」（照片模糊、看不到車架）	▲ 待確認
4	兩者都可用，而且都合格	● 合格
5	兩者都可用，而且都不合格	■ 不合格
6	兩者都可用，但意見不同	▲ 待確認
7	只有 Claude 可用（自建模型信心 < 60%）	採用 Claude 結果
8	只有自建模型可用（Claude 連不上或沒設定）	信心 ≥ 85% 且不合格 → ■；其他 → ▲
9	兩者都無法判斷	▲ 待確認

「可用」的意思：自建模型的最高機率類別不是 X，而且信心 ≥ LOCAL_MIN_CONFIDENCE（預設 60%）。最終分數 = Claude 分數 × 60% + 自建模型分數 × 40%（兩者都可用時），損耗率 = 100 - 最終分數。website/model/ 沒有模型時，結果和只用 Claude 完全相同。

合併程式

website/script.js — combineResults()

```
// Claude 發現的硬性問題（自建模型沒學過的少見狀況），Claude 有否決權
function isHardFail(c) {
    return c.verdict === "fail" && (
        c.bikeType !== "standard" ||
        c.components?.frame?.condition === "severe" ||
        c.components?.wheels_tires?.condition === "severe");
}
```

```
// 合併規則：由上往下檢查，符合第一條就停止。回傳 null 代表兩邊都沒有結果
function combineResults(local, claude) {
    if (!local && !claude) return null;
    if (!local) return claude; // 沒有自建模型：與原本只用 Claude 的結果相同

    const localOk = !local.notBike && local.confidence >= LOCAL_MIN_CONFIDENCE;
    const localPass = localOk && local.score >= PASS_SCORE;
    let score = null;
    if (claude && localOk) score = Math.round(claude.score * CLAUDE_WEIGHT + local.score * (1 - CLAUDE_WEIGHT
        ));
    else if (claude) score = claude.score;
    else if (localOk) score = local.score;

    let verdict, reasonKey;
    if (claude && isHardFail(claude)) {
        verdict = "fail"; reasonKey = "hy_hard_fail";
    } else if (local.notBike && local.confidence >= 0.7) {
        verdict = "review"; reasonKey = "hy_local_notbike";
    } else if (claude && claude.verdict === "review") {
        verdict = "review"; reasonKey = "hy_claude_review";
    } else if (claude && localOk) {
        const claudePass = claude.verdict === "pass";
        if (claudePass === localPass) { verdict = claudePass ? "pass" : "fail"; reasonKey = "hy_agree"; }
        else { verdict = "review"; reasonKey = "hy_disagree"; }
    } else if (claude) {
        verdict = claude.verdict; reasonKey = "hy_claude_only";
    } else if (localOk) {
        verdict = (!localPass && local.confidence >= 0.85) ? "fail" : "review"; reasonKey = "hy_local_only";
    } else {
        verdict = "review"; reasonKey = "hy_none";
    }
    return {
        hybrid: true, verdict, score, wearRate: score === null ? null : 100 - score,
        passScore: PASS_SCORE, reasonKey, local, claude, assessedAt: Date.now()
    };
}
```

上傳流程

在 **confirmBikeUpload()** 裡，兩個模型**同時**執行，總時間只取決於比較慢的 Claude（約 10–30 秒）：

website/script.js — confirmBikeUpload()

```
// 自建模型與 Claude 同時執行，總時間只取決於比較慢的那個
const useClaude = !!AI_INSPECT_URL;
if (useClaude) toast(t("ai_checking"), "info", 6000);
const [localRes, claudeRes] = await Promise.allSettled([
    runLocalModel(photos),
    useClaude ? runAiInspection(photos) : Promise.resolve(null)
]);
const local = localRes.status === "fulfilled" ? localRes.value : null;
```

```
const claude = claudeRes.status === "fulfilled" ? claudeRes.value : null;
if (localRes.status === "rejected") console.warn("自建模型檢測失敗：", localRes.reason);
if (claudeRes.status === "rejected") {
    console.warn("Claude 檢測失敗：", claudeRes.reason);
    toast(t("ai_unavailable"), "warn", 6000);
}
ai = combineResults(local, claude);
```

存進資料庫的內容

每筆回收申請的 **ai** 欄位會保存完整結果，第 9 章分析準確度時會用到：

欄位	內容
verdict / score / wearRate	最終結果：pass、fail、review，以及合併後的分數與損耗率
reasonKey	採用了哪一條合併規則（例如 hy_agree、hy_disagree）
local	自建模型結果：各等級機率 probs、最可能的等級 topClass、信心 confidence、分數 score
claude	Claude 結果：逐部位分數與說明（和只用 Claude 時的格式相同）

技師後台不用另外設定

後台的「AI 68 · 待確認」標籤會顯示合併後的結果；點下去可以看到兩個模型各自的分數、自建模型的各等級機率，以及 Claude 的逐部位分析。

第 9 章 測試與調整

用數字確認模型真的有用，再決定門檻與權重。

看懂混淆矩陣

把 test/ 的照片逐一丟給模型，記錄「正確答案」與「模型答案」，就能畫出混淆矩陣。對角線（粗體）是答對的數量，其他格子是搞混的情況：

正確答案 \ 模型答案	A	B	C	D	X	準確率
A	**17**	3	0	0	0	85%
B	2	**15**	3	0	0	75%
C	0	4	**14**	2	0	70%
D	0	0	2	**18**	0	90%
X	0	0	0	1	**19**	95%

從這個範例可以看出：**B 和 C 最常搞混**（4 + 3 張），正好是合格門檻附近，這也是最需要 Claude 和技師把關的地方。改善方法是補更多 B、C 的照片，並把兩者的判斷標準寫得更清楚。

驗收標準（建議）

指標	最低要求	說明
整體準確率	≧ 80%	低於這個數字，先改善資料再上線
D 被判成 A/B 的比例	≦ 5%	最危險的錯誤：壞車被當好車
合併後的「待確認」比例	≦ 30%	太高代表技師工作量沒有減少，需要提升自建模型
合併結果與技師最終判斷一致	≧ 90%	上線後每月統計一次

可以調整的地方

想要的效果	調整什麼	在哪裡
更嚴格（少收壞車）	提高 PASS_SCORE，例如 75	wrangler.toml 與 script.js 兩處都要改
更寬鬆（多收車）	降低 PASS_SCORE，例如 65	同上
減少技師工作量	降低 LOCAL_MIN_CONFIDENCE；自建模型權重調高	script.js
減少誤判	提高 LOCAL_MIN_CONFIDENCE；讓更多案例進入待確認	script.js
模型更準	補照片、修正標註後重新訓練	Teachable Machine

持續改進：每月重新訓練

1. 從技師後台匯出上個月已審核的申請照片（通過 → 依技師判斷放 A/B/C；退回 → 依原因放 C/D/X）。
2. 特別收集「合併結果和技師判斷不同」的案例，這些是模型最需要學的。
3. 把照片加進 Teachable Machine 專案（從 Google 雲端硬碟開啟），重新訓練並用 test/ 驗收。
4. 準確率有進步才替換 website/model/ 的檔案；記得保留舊版本，有問題可以換回來。

第 10 章 常見問題與上線檢查

遇到問題先看這裡。

常見問題

問題	可能原因	解決方法
訓練準確率很高，但測試很差	過度擬合；同一台車同時出現在訓練與測試	減少 Epochs；重新分配測試照片，確保同一台車只在一邊
模型總是猜同一個類別	類別照片數量差太多	補少數類別的照片，讓各類別數量接近
模型好像在看背景	某類別的照片都在同一個地方拍	每個類別都在多種場景拍攝
結果視窗沒有「自建模型」區塊	model/ 路徑或檔名錯誤，或直接雙擊 HTML 開啟	確認三個檔案都在 website/model/，並用本機伺服器開啟
主控台出現「模型類別必須是 A、B、C、D、X」	Teachable Machine 的類別名稱不是 A–X	回到第 6 章把類別改名後重新訓練、匯出
主控台出現「無法載入 https://cdn...」	網路連不到函式庫網站	檢查網路；或把兩個函式庫下載到網站資料夾，修改 TFJS_URL、TM_IMAGE_URL
Claude 回傳 401 unauthorized	學生沒登入，或 Firebase token 過期	重新登入後再試
Claude 回傳 403 origin_not_allowed	網站網址不在允許清單	把網址加入 wrangler.toml 的 ALLOWED_ORIGINS，重新部署
Claude 回傳 429 rate_limited	短時間內呼叫太多次	稍後再試；必要時到 Console 提高用量上限
預標註程式顯示 API Key 無效	環境變數沒設定或複製錯誤	重新執行 $env:ANTHROPIC_API_KEY = "..."

上線前檢查清單

- ☐ 第 3 章的等級定義已經和技師確認過
- ☐ 每個類別至少 50 張照片，而且都經過人工覆核
- ☐ test/ 測試照片沒有被拿去訓練，整體準確率 ≧ 80%
- ☐ website/model/ 有 model.json、metadata.json、weights.bin
- ☐ Claude 後端已部署，AI_INSPECT_URL 已填入，ALLOWED_ORIGINS 包含正式網址

☐	Claude Console 已設定每月花費上限
☐	PASS_SCORE 在 wrangler.toml 與 script.js 兩處一致；上傳測試時結果視窗有「自建模型」區塊
☐	用 A、C、D、X 各一組照片實際走過申請流程，結果符合預期
☐	技師知道「待確認」的案例要特別仔細看

重要提醒：AI 是輔助，技師才是最後決定者

不論模型多準，都會有判斷錯誤的時候。系統的設計是讓 AI 擋掉明顯不合格的車、減少技師的工作量，但所有申請最後都還是由技師核准。另外，AI 結果是由學生的瀏覽器送出並存進資料庫，懂技術的人有可能竄改，這也是保留技師審核的原因之一。